

```
import re

text_data = [
    "machine learning models learn patterns from data.",
    "sequence models process data step by step.",
    "recurrent neural networks are designed for sequential tasks.",
    "rnn models maintain hidden states across time steps.",
    "long short term memory networks solve long dependency problems.",
    "lstm uses gates to control information flow.",
    "gru models simplify the lstm architecture.",
    "sequence prediction is useful in many applications.",
    "language modeling predicts the next word in a sentence.",
    "speech recognition processes audio sequences.",
    "time series forecasting predicts future values.",
    "music generation creates new melodies.",
    "generative models learn probability distributions.",
    "they generate new samples similar to training data.",
    "sequence generation is widely used in artificial intelligence.",
    "deep learning improves sequence modeling performance."
]

processed_data = [re.sub(r'[^\w\s]', '', sentence.lower()) for sentence in text_data]
```

```
from collections import Counter

words = " ".join(processed_data).split()
word_counts = Counter(words)
vocab = sorted(word_counts, key=word_counts.get, reverse=True)
vocab_size = len(vocab)

word_to_int = {word: i for i, word in enumerate(vocab)}
int_to_word = {i: word for i, word in enumerate(vocab)}
```

```
import torch

seq_length = 4
X = []
y = []

for sentence in processed_data:
    tokens = sentence.split()
    if len(tokens) <= seq_length:
        continue
    for i in range(len(tokens) - seq_length):
        X.append([word_to_int[w] for w in tokens[i:i+seq_length]])
        y.append(word_to_int[tokens[i+seq_length]])

X = torch.tensor(X, dtype=torch.long)
y = torch.tensor(y, dtype=torch.long)
```

```
import torch.nn as nn

class LSTMGenerator(nn.Module):
    def __init__(self, vocab_size, embed_size, hidden_size):
        super(LSTMGenerator, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.lstm = nn.LSTM(embed_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x):
        x = self.embedding(x)
        out, _ = self.lstm(x)
        out = self.fc(out[:, -1, :])
        return out
```

```
model = LSTMGenerator(vocab_size, 32, 64)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

epochs = 100
for epoch in range(epochs):
    optimizer.zero_grad()
    outputs = model(X)
    loss = criterion(outputs, y)
    loss.backward()
    optimizer.step()
```

```
def generate_sequence(model, seed_words, num_words):
    model.eval()
    current_words = seed_words.split()
    for _ in range(num_words):
        x_input = torch.tensor([[word_to_int.get(w, 0) for w in current_words[-seq_length:]]], dtype=torch.long)
        with torch.no_grad():
            output = model(x_input)
            predicted_idx = torch.argmax(output, dim=1).item()
            predicted_word = int_to_word[predicted_idx]
            current_words.append(predicted_word)
    return " ".join(current_words)

seed = "machine learning models learn"
print(generate_sequence(model, seed, 5))
```

machine learning models learn patterns from data data step